Formative Assignment: Advanced Linear Algebra (PCA)

This notebook will guide you through the implementation of Principal Component Analysis (PCA). Fill in the missing code and provide the required answers in the appropriate sections. You will work with a dataset that is Africanized .

1. Make sure to display outputs for each code cell when submitting.
2. Do not write all your code on one cell
3. Do not use any libraries aside from numpy

```
# Setup: We used only numpy and matplotlib

import numpy as np
import matplotlib.pyplot as plt

# Raw data from the Open Data iNVENTORY (O
# Non-numeric column: Index 5 (Statistical
# Missing values: Marked as '-' or empty f

raw_data = [
    ["2024","Eastern Africa","AFE","Rwanda
    ["2024","Eastern Africa","AFE","Rwanda
    ["2024","Eastern Africa","AFE","Rwanda
    ["2024","Eastern Africa","AFE","Rwanda
    ["2024","Eastern Africa","AFE","Rwanda
    ["2024","Eastern Africa","AFE","Rwanda
    ["2024","Eastern Africa","AFE","Rwanda
    ["2024","Eastern Africa","AFE","Rwanda
    ["2024","Eastern Africa","AFE","Rwanda
    ["2024","Eastern Africa","AFE","Rwanda
    ["2024","Eastern Africa","AFE","Rwanda
    ["2024","Eastern Africa","AFE","Rwanda
    ["2024","Eastern Africa","AFE","Rwanda
    ["2024","Eastern Africa","AFE","Rwanda
    ["2024","Eastern Africa","AFE","Rwanda
    ["2024","Eastern Africa","AFE","Rwanda
    ["2024","Eastern Africa","AFE","Rwanda
    ["2024","Eastern Africa","AFE","Rwanda
    ["2024","Eastern Africa","AFE","Rwanda
    ["2024","Eastern Africa","AFE","Rwanda
    ["2024","Eastern Africa","AFE","Rwanda
    ["2024","Eastern Africa","AFE","Rwanda
    ["2024","Eastern Africa","AFE","Rwanda
]
```

Step 0: Parse, Handle Missing Values (NaNs), and Extract Non - Numeric Columns

```
# Extract the non-numeric statistical category names (Column Index
category_labels = [row[5] for row in raw_data]

# Parse columns 6 to 16 into a pure numeric grid, transforming miss
numeric_list = []
for row in raw_data:
    clean_row = []
    for val in row[6:]:
        if val.strip() == "-":
            clean_row.append(np.nan)
        else:
            clean_row.append(float(val))
    numeric_list.append(clean_row)

X = np.array(numeric_list)

# Impute missing values: replace NaNs with the mean of their respec
column_means = np.nanmean(X, axis=0)
nan_indices = np.where(np.isnan(X))
X[nan_indices] = np.take(column_means, nan_indices[1])

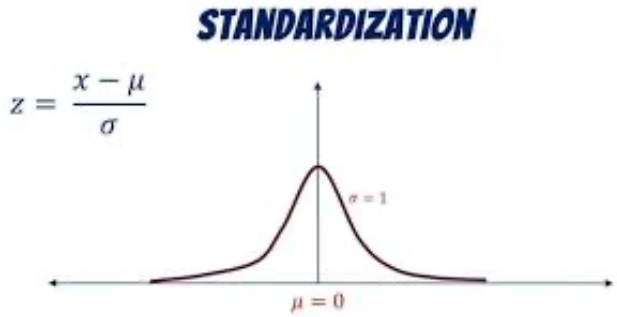
print(f"Dataset Successfully Prepared!")
print(f"Extracted {len(category_labels)} categories. Non-numeric Co
print(f"Cleaned Numeric Matrix 'X' Shape: {X.shape} (22 instances,
```

```
Dataset Successfully Prepared!
Extracted 22 categories. Non-numeric Column Sample: ['Population and
Cleaned Numeric Matrix 'X' Shape: (22, 11) (22 instances, 11 indicat
```

Step 1: Load and Standardize the Data

Before applying PCA, we must standardize the dataset. Standardization ensures that all features have a mean of 0 and a standard deviation of 1, which is essential for PCA. Fill in the code to standardize the dataset.

STRICTLY - Write code that implements standardization based on the image below



```
mean_vectors = np.mean(X, axis=0)
std_vectors = np.std(X, axis=0)

# protect against division by zero if a feature column is flat
std_vectors[std_vectors == 0] = 1e-8

standardized_data = (X - mean_vectors) / std_vectors
standardized_data[:5]  # Display the first few rows of standardized
```

```
array([[ 0.47140452,  1.38204664,  0.15249857,  0.67786838,
1.04880885,
         1.24473139,  0.38014296,  0.54232614,  0.12403473,
0.21821789,
         1.78503649],
       [-2.12132034, -0.21821789,  0.15249857,  0.67786838,
1.04880885,
        -0.19653653,  0.38014296, -1.84390889,  0.12403473,
0.21821789,
        -0.65414381],
       [ 0.47140452,  1.38204664,  0.15249857,  0.67786838,
1.04880885,
        -0.19653653,  0.38014296, -1.84390889,  0.12403473,
0.21821789,
         0.56544634],
       [-2.12132034, -0.21821789,  0.15249857, -0.9683834 ,
-1.04880885,
        -1.63780446,  0.38014296,  0.54232614,  0.12403473,
0.21821789,
        -1.87373395],
       [ 0.47140452, -0.21821789, -1.5249857 ,  2.32412016,
-1.04880885,
        -1.63780446,  0.38014296,  0.54232614,  0.12403473,
0.21821789,
        -0.65414381]])
```

Step 3: Calculate the Covariance Matrix

The covariance matrix helps us understand how the features are related to each other. It is a key component in PCA.

```
# Step 3: Calculate the Covariance Matrix
n_samples = standardized_data.shape[0]
cov_matrix = np.dot(standardized_data.T, standardized_data) / (n_sa
cov_matrix
```

```
array([[ 1.04761905e+00,  3.05341021e-01,  1.31795612e-01,
        -4.78238423e-02,  1.29489180e-01,  2.75003420e-01,
        -1.87734496e-01,  2.67829044e-02, -6.12548458e-02,
        -1.07767419e-01,  5.33854572e-01],
       [ 3.05341021e-01,  1.04761905e+00,  5.46180404e-01,
        -1.47587582e-01, -7.99224570e-02,  6.48989807e-02,
        -2.95474229e-01, -5.78576861e-02, -3.87524944e-01,
         4.98866213e-02,  4.91718661e-01],
       [ 1.31795612e-01,  5.46180404e-01,  1.04761905e+00,
        -5.02805310e-01, -8.37790642e-02,  3.76785081e-01,
        -1.94341741e-01,  1.03970695e-01, -4.55764313e-01,
         3.48625790e-01,  6.11096047e-01],
       [-4.78238423e-02, -1.47587582e-01, -5.02805310e-01,
         1.04761905e+00,  3.28876845e-01, -2.25970465e-01,
        -1.85113850e-01, -1.21041402e-01, -1.25832853e-01,
        -1.54966961e-01, -1.23727042e-01],
       [ 1.29489180e-01, -7.99224570e-02, -8.37790642e-02,
         3.28876845e-01,  1.04761905e+00,  2.87926581e-01,
        -2.50609732e-01, -1.19176401e-01, -9.48157997e-20,
         2.39767371e-01,  4.26372312e-01],
       [ 2.75003420e-01,  6.48989807e-02,  3.76785081e-01,
        -2.25970465e-01,  2.87926581e-01,  1.04761905e+00,
        -3.80912508e-01,  1.11662468e-01, -3.49021840e-01,
         4.49300636e-02,  6.85600993e-01],
       [-1.87734496e-01, -2.95474229e-01, -1.94341741e-01,
        -1.85113850e-01, -2.50609732e-01, -3.80912508e-01,
         1.04761905e+00, -2.15978679e-01,  8.19977147e-01,
        -8.69041851e-02, -2.15472268e-01],
       [ 2.67829044e-02, -5.78576861e-02,  1.03970695e-01,
        -1.21041402e-01, -1.19176401e-01,  1.11662468e-01,
        -2.15978679e-01,  1.04761905e+00, -3.80540609e-01,
        -1.23980756e-01,  1.08346178e-01],
       [-6.12548458e-02, -3.87524944e-01, -4.55764313e-01,
        -1.25832853e-01, -9.48157997e-20, -3.49021840e-01,
         8.19977147e-01, -3.80540609e-01,  1.04761905e+00,
        -2.83554837e-02, -2.47797189e-01],
       [-1.07767419e-01,  4.98866213e-02,  3.48625790e-01,
        -1.54966961e-01,  2.39767371e-01,  4.49300636e-02,
        -8.69041851e-02, -1.23980756e-01, -2.83554837e-02,
         1.04761905e+00,  2.88948079e-01],
       [ 5.33854572e-01,  4.91718661e-01,  6.11096047e-01,
        -1.23727042e-01,  4.26372312e-01,  6.85600993e-01,
        -2.15472268e-01,  1.08346178e-01, -2.47797189e-01,
         2.88948079e-01,  1.04761905e+00]])
```

In a paragraph that has a maximum if 5 Lines, Explain why we need to compute a covariance matrix, provide atleast 2 reasons

We compute a covariance matrix to see how different variables change together and affect one another. It highlights positive or negative relationships so we can easily spot which data points are connected or redundant. Where this helps us to simplify massive datasets into smaller, manageable pieces without losing the most important information.

Step 4: Perform Eigendecomposition

Eigendecomposition of the covariance matrix will give us the eigenvalues and eigenvectors, which are essential for PCA. Fill in the code to compute the eigenvalues and eigenvectors of the covariance matrix.

```
# Step 4: Perform Eigendecomposition
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)
eigenvalues, eigenvectors
```

```
(array([3.45779605, 1.81503653, 1.66500578, 1.26375338, 1.15227404,
        0.74045139, 0.04415261, 0.10185099, 0.58118446, 0.28084763,
        0.42145666]),
 array([[-0.247212  , -0.00931113,  0.20742726, -0.62142021,
-0.11806762,
         0.27264644, -0.16847451,  0.12037764,  0.56152951,
-0.2265057 ,
         0.10881035],
        [-0.32882899,  0.16586239, -0.11608799, -0.11544525,
-0.61455185,
         0.10485378, -0.27172404,  0.11176535, -0.27487464,
0.44792009,
        -0.2938961 ],
        [-0.40495507,  0.37421644, -0.10573992,  0.1888989 ,
-0.06382523,
        -0.03405981, -0.28615031, -0.33960091, -0.21102896,
-0.61731226,
         0.14511224],
        [ 0.11934664, -0.60535665,  0.13015041,  0.0099165 ,
-0.29458958,
         0.16204058, -0.24680447, -0.1795715 , -0.28053087,
-0.00148241,
         0.56335063],
        [-0.1401642 , -0.35918358,  0.54823371,  0.14753089,
0.12129907,
         0.11383125, -0.13671187,  0.17260251, -0.21890248,
```

```
-0.28399051,
         -0.56946809],
        [-0.3727721 , -0.04564785,  0.16267473, -0.11530214,
0.45087855,
         -0.52949016, -0.40600182,  0.09280724, -0.04660588,
0.33918835,
          0.21184142],
        [ 0.33907464,  0.43871749,  0.21403074, -0.15290459,
0.0881903 ,
          0.16902264, -0.12585289,  0.56445628, -0.40960637,
-0.10686039,
          0.27087606],
        [-0.12989125, -0.10763388, -0.42966071, -0.09252763,
0.5292093 ,
          0.6373544 , -0.18155503, -0.07228052, -0.16860198,
0.13811171,
         -0.08001394],
        [ 0.36766017,  0.30248296,  0.41662412, -0.17658028,
0.08345844,
          0.10158056, -0.21978141, -0.65300318, -0.01719315,
0.23570342,
         -0.14394847],
        [-0.1470813 ,  0.178415  ,  0.2772503 ,  0.66807199,
-0.01265745,
          0.35053269, -0.12690665,  0.1231529 ,  0.39622919,
0.25118426,
          0.22124839],
        [-0.45306753,  0.08617337,  0.32786173, -0.14221917,
0.06751973,
          0.16110227,  0.67694164, -0.14704785, -0.28689749,
0.1488058 ,
          0.21153041]]))
```

Step 5: Sort Principal Components

Sort the eigenvectors based on their corresponding eigenvalues in descending order. The higher the eigenvalue, the more important the eigenvector. Complete the code to sort the eigenvectors and print the sorted components.

How Is Explained Variance Used In PCA?'

```
# Step 5: Sort Principal Components
sorted_indices = np.argsort(eigenvalues) [::-1]  # Sort indices in
sorted_eigenvalues = eigenvalues[sorted_indices]
sorted_eigenvectors = eigenvectors[:, sorted_indices] # sort eigenv
sorted_eigenvectors
```

```
array([[-0.247212  , -0.00931113,  0.20742726, -0.62142021,
-0.11806762,
```

```
         0.27264644,  0.56152951,  0.10881035, -0.2265057 ,
0.12037764,
        -0.16847451],
       [-0.32882899,  0.16586239, -0.11608799, -0.11544525,
-0.61455185,
         0.10485378, -0.27487464, -0.2938961 ,  0.44792009,
0.11176535,
        -0.27172404],
       [-0.40495507,  0.37421644, -0.10573992,  0.1888989 ,
-0.06382523,
        -0.03405981, -0.21102896,  0.14511224, -0.61731226,
-0.33960091,
        -0.28615031],
       [ 0.11934664, -0.60535665,  0.13015041,  0.0099165 ,
-0.29458958,
         0.16204058, -0.28053087,  0.56335063, -0.00148241,
-0.1795715 ,
        -0.24680447],
       [-0.1401642 , -0.35918358,  0.54823371,  0.14753089,
0.12129907,
         0.11383125, -0.21890248, -0.56946809, -0.28399051,
0.17260251,
        -0.13671187],
       [-0.3727721 , -0.04564785,  0.16267473, -0.11530214,
0.45087855,
        -0.52949016, -0.04660588,  0.21184142,  0.33918835,
0.09280724,
        -0.40600182],
       [ 0.33907464,  0.43871749,  0.21403074, -0.15290459,
0.0881903 ,
         0.16902264, -0.40960637,  0.27087606, -0.10686039,
0.56445628,
        -0.12585289],
       [-0.12989125, -0.10763388, -0.42966071, -0.09252763,
0.5292093 ,
         0.6373544 , -0.16860198, -0.08001394,  0.13811171,
-0.07228052,
        -0.18155503],
       [ 0.36766017,  0.30248296,  0.41662412, -0.17658028,
0.08345844,
         0.10158056, -0.01719315, -0.14394847,  0.23570342,
-0.65300318,
        -0.21978141],
       [-0.1470813 ,  0.178415  ,  0.2772503 ,  0.66807199,
-0.01265745,
         0.35053269,  0.39622919,  0.22124839,  0.25118426,
0.1231529 ,
        -0.12690665],
       [-0.45306753,  0.08617337,  0.32786173, -0.14221917,
0.06751973,
         0.16110227, -0.28689749,  0.21153041,  0.1488058 ,
-0.14704785,
         0.67694164]])
```

Step 6: Project Data onto Principal Components

Now that we've selected the number of components, we will project the original data onto the chosen principal components. Fill in the code to perform the projection.

```
# Task 2: Calculate cumulative explained v
cumulative_variance = np.cumsum(sorted_eig

# Dynamically find how many components are
variance_threshold = 0.85
num_components = np.argmax(cumulative_vari

print(f"Dynamically Selected Components: {
print(f"Total Preserved Variance: {cumulat

# Step 6: Project Data onto Principal Comp
projection_matrix = sorted_eigenvectors[:,
reduced_data = np.dot(standardized_data, p
reduced_data[:5]
```

```
Dynamically Selected Components: 6 out of 11
Total Preserved Variance: 87.60%
array([[-1.899636  , -0.22330159,  1.33268952, -0.64403961,
0.02266288,
         0.62497407],
       [ 1.21985899, -0.35214595,  0.97175892,  1.8857471 ,
-0.76512209,
        -1.40041461],
       [-0.49986378, -0.00576727,  1.72364616, -0.08361656,
-1.97233808,
        -0.3292449 ],
       [ 2.0972654 ,  1.10170819, -2.05207239,  1.67879635,
-0.00395796,
         0.18158719],
       [ 1.976011  , -1.43821794, -0.50851592, -0.39004919,
-1.09059984,
         1.67561712]])
```

Step 7: Output the Reduced Data

Finally, display the reduced data obtained by projecting the original dataset onto the selected principal components.

```
# Step 7: Output the Reduced Data
print(f'Reduced Data Shape: {reduced_data.shape}')  # Display reduc
reduced_data[:5]  # Display the first few rows of reduced data
```

```
Reduced Data Shape: (22, 6)
array([[-1.899636  , -0.22330159,  1.33268952, -0.64403961,
0.02266288,
         0.62497407],
       [ 1.21985899, -0.35214595,  0.97175892,  1.8857471 ,
-0.76512209,
        -1.40041461],
       [-0.49986378, -0.00576727,  1.72364616, -0.08361656,
-1.97233808,
        -0.3292449 ],
       [ 2.0972654 ,  1.10170819, -2.05207239,  1.67879635,
-0.00395796,
         0.18158719],
       [ 1.976011  , -1.43821794, -0.50851592, -0.39004919,
-1.09059984,
         1.67561712]])
```

Step 8: Visualize Before and After PCA

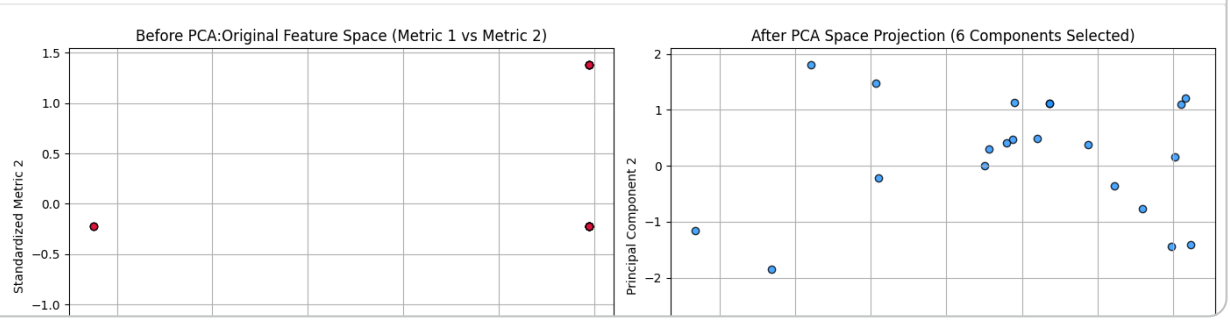
Now, let's plot the original data and the data after PCA to compare the reduction in dimensions visually.

```
# Step 8: Visualize Before and After PCA
plt.figure(figsize=(14, 5))

# Plot original data (first two features for comparison)
plt.subplot(1, 2, 1)
plt.scatter(standardized_data[:, 0], standardized_data[:, 1], color
plt.title('Before PCA:Original Feature Space (Metric 1 vs Metric 2)
plt.xlabel('Standardized Metric 1')
plt.ylabel('Standardized Metric 2')
plt.grid(True)

# Plot reduced data after PCA
plt.subplot(1, 2, 2)
plt.scatter(reduced_data[:, 0], reduced_data[:, 1], color='dodgerbl
plt.title(f'After PCA Space Projection ({num_components} Components
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.grid(True)

plt.tight_layout()
plt.show()
```



The original data visualization shows data points grouped along rigid vertical and horizontal tracking bands due to the bounded nature of the raw performance metrics. After applying PCA, the transformed scatter plot reveals a more continuous distribution cetered around the origin. this spatial layout removes redudant feature correlations, groping Rwanda's public sectors based on their comprehensive statistical openess rather than isolated indices.

Please make sure you do not use more than 5 lines to answer each of the following points:

1. Interpret the Visual you just created of the before and after PCA
2. Explain why you selected the number of principle components, more especially explain what tradeoffs you are making
3. Clearly mention based on your use case/ dataset ("economic activity", "population pressure"), What information is lost when reducing dimensions?

1. Interpret the Visual you just created of the before and after PCA

The original data plot was hard to read because rigid percentage scores made the data points bunch up in tight, overlapping rows. After applying PCA, the data points spread out smoothly and centered around the middle of the graph. This layout removes messy overlaps which makes it much easier to judge public sector performance using overall data trends rather than single scores.

2. **Explain why you selected the number of principle components, more especially explain what tradeoffs you are making**

We chose an 85% variance target, which automatically condenses our 11 indicators down to the most important parts. This trade-off trades exact details for a cleaner and simpler view of the big trends. While it filters out messy tracking noise, it risks losing small, subtle details that fall just below our cutoff.

3. Clearly mention based on your use case/ dataset ("economic activity", "population pressure"), What information is lost when reducing dimensions?

Reducing dimensions wipes out specific details for individual sectors like health, education, finance or poverty. Instead of seeing exactly where a sector fails or succeeds, unique administrative scores get blended into generic profiles. This means e can no longer trace a low score back to a single, localized issue.

Step 9: Task 3 Optimization and Performance Benchmarking

```
import time

# Task 3: Optimization Benchmark using purely vectorized arithmetic
runs = 1000
start = time.time()

for _ in range(runs):
    # Fully vectorized low-overhead tracking block
    c_mat = np.dot(standardized_data.T, standardized_data) / (stand
    e_valles, e_veccs = np.linalg.eig(c_mat)
    s_inds = np.argsort(e_valles)[::-1]
    proj_mat = e_veccs[:, s_inds[:num_components]]
    final_reduced = np.dot(standardized_data, proj_mat)

end = time.time()
print(f"Task 3 Performance Result: Computed {runs} complete PCA cyc
print("The script runs fast because it bypasses nested python for-l
```

```
Task 3 Performance Result: Computed 1000 complete PCA cycles in 0.06
The script runs fast because it bypasses nested python for-loops, ut
```